Login.cshtml Razor Page, which reads the email and password from the request using model binding.

The meaty work happens inside the `SignInManager` service. This is responsible for loading a user entity with the provided username from the database and validating that the password they provided is correct.

WARNING Never store passwords in the database directly. They should be *hashed* using a strong one-way algorithm. The ASP.NET Core Identity system does this for you, but it's always wise to reiterate this point!

If the password is correct, `SignInManager` creates a new `ClaimsPrincipal` from the user entity it loaded from the database and adds the appropriate claims, such as the email address. It then replaces the old, anonymous `HttpContext.User` principal with the new, authenticated principal.

Finally, `SignInManager` serializes the principal, encrypts it, and stores it as a cookie. A *cookie* is a small piece of text that's sent back and forth between the browser and your app along with each request, consisting of a name and a value.

This authentication process explains how you can set the user for a request when they first log in to your app, but what about subsequent requests? You send your password only when you first log in to an app, so how does the app know that it's the same user making the request?

Authenticating users for subsequent requests

The key to persisting your identity across multiple requests lies in the final step of figure 23.2, where you serialized the principal in a cookie. Browsers automatically send this cookie with all requests made to your app, so you don't need to provide your password with every request.